PROJECT REPORT

REAL-TIME VIRTUAL INVISIBILITY CLOAK

Using Computer Vision and Hand Gesture Recognition

Course: Image and Video Processing

Student Name(s)	Vaidik Agrawal, Shresth Sambharia, Punya Chopra
Roll Number(s)	F033, F028 ,
Programme / Semester / Division	B.Tech (C.E.), SEM - V, Section - A
Faculty Name	Dr. Raj Gaurav Mishra Sir
Institute / Department	STME
Academic Year	2026–2027

Submitted in partial fulfilment of the requirements for the course

Contents

- Abstract

1. 1. Introduction
2. 2. Problem Statement and Objectives
3. 3. Scope of the Project
4. 4. Dataset / Input Description
5. 5. Methodology / Proposed Approach
6. 6. Techniques and Algorithms Used
7. 7. Implementation
8. 8. Results and Discussion
9. 9. Challenges and Limitations
10. 10. Conclusion and Future Work

- References

Note: Replace the student/institute placeholders on the title page before submission. Performance figures are intentionally not fabricated; quantitative measurements should be added after testing on the target machine.

Abstract

This project implements a real-time virtual invisibility cloak using a webcam and classical image/video processing. The aim is to replace pixels belonging to a selected-colour cloth with pixels from a previously captured background, creating the visual impression that the cloth-covered region is transparent. The application captures live frames, converts them to HSV colour space, thresholds a selected colour, applies morphological filtering to reduce isolated noise, and compares the current frame with the reference background to suppress some static-background detections. A soft mask is used to blend the captured background into the live frame. MediaPipe hand landmarks provide a gesture-based interface: the index fingertip controls the cursor and a thumb–index pinch activates on-screen controls for colour selection, background capture, cloak toggling, and mask preview. The prototype demonstrates the core invisibility-cloak effect and hands-free interaction using Python, OpenCV, NumPy, and MediaPipe. The result is qualitative and depends on lighting, camera stability, cloth colour, and background contrast. Formal accuracy, latency, and frame-rate measurements have not yet been recorded.

Keywords: Image processing, video processing, HSV segmentation, background subtraction, morphological filtering, hand gesture recognition, OpenCV, MediaPipe.

1. Introduction

1.1 Background

An invisibility-cloak effect is a visual compositing technique in which a selected region of a live camera frame is replaced with a corresponding region from a reference background. The effect does not make an object physically invisible; it creates an illusion by combining pixels from two images. A stable camera and a background captured before the subject enters the scene are central to the method.

1.2 Relevance of Image and Video Processing

The project demonstrates several core concepts from image and video processing: frame acquisition, colour-space conversion, threshold-based segmentation, binary-mask processing, frame differencing, morphological operations, temporal interaction, and image compositing. Because processing is performed continuously on webcam frames, the project also illustrates practical issues such as lighting variation, noise, latency, and temporal instability.

1.3 Motivation

The project was selected because it turns abstract image-processing operations into an immediately observable effect. It combines a visual output with a hands-free control interface, making it suitable for a classroom demonstration of classical computer vision and basic gesture-based interaction. The implementation also exposes the limitations of colour-only segmentation, especially when the cloth and background have similar colours.

1.4 Project Overview

The application allows the user to select red, blue, green, or white as the target cloth colour. A background reference is captured from the webcam. Each incoming frame is segmented by colour and, when a reference background is available, filtered using frame differences. The resulting mask is softened and used to blend the background into the detected region. MediaPipe Hands supplies hand landmarks for moving a cursor and selecting interface controls with a pinch gesture.

2. Problem Statement and Objectives

2.1 Problem Statement

Develop a real-time webcam application that creates an invisibility-cloak illusion by detecting a selected-colour cloth and replacing its image region with a captured background, while providing a hand-gesture interface for selecting options and controlling the effect.

2.2 Objectives

- Acquire and display live video frames from a webcam at a consistent processing resolution.
- Segment a user-selected red, blue, green, or white cloth using HSV colour thresholds.
- Reduce segmentation noise through morphological filtering and suppress some static-background detections through frame differencing.
- Blend the reference background into the detected cloth region using a soft alpha mask.
- Use hand landmarks and a thumb–index pinch gesture to interact with on-screen controls without relying exclusively on a mouse.

3. Scope of the Project

3.1 Included

- Real-time webcam input and a captured reference background.
- HSV threshold-based colour selection for four target colours.
- Binary mask generation, morphological opening/closing, Gaussian smoothing, and background compositing.
- MediaPipe Hands landmark-based cursor movement and pinch-to-select interaction.
- Keyboard fallbacks for background capture, cloak toggle, mask preview, sensitivity adjustment, and exit.

3.2 Excluded

- Physical invisibility or reconstruction of unseen body surfaces.
- Robust semantic segmentation of arbitrary cloth regardless of colour.
- Automatic compensation for a moving camera or a changing room background.
- A trained custom machine-learning model for cloth segmentation.
- A benchmarked real-time performance guarantee across different hardware.

3.3 Application Domain

The project is an educational prototype for image/video processing and human–computer interaction. Its techniques are relevant to basic visual effects, colour-based object tracking demonstrations, camera filters, and interactive computer-vision prototypes. It is not intended for safety-critical or production visual-effects use.

4. Dataset / Input Description

No fixed offline dataset is used in the current implementation. The input is a live webcam stream, and the reference background is captured at run time. Therefore, a dataset name, total image count, and fixed video duration are not applicable.

Input property	Description
Source	Live webcam stream
Configured frame size	960 × 540 pixels in the current code
Frame format	OpenCV BGR image array; converted to RGB for MediaPipe and HSV for colour segmentation
Reference background	A frame captured from the empty scene before the subject enters

Target object	A cloth matching the selected RED, BLUE, GREEN, or WHITE threshold range
Number of frames / duration	Variable; depends on the length of the live session
Lighting and scene	Must be sufficiently stable for reliable colour thresholding and frame differencing

For a reproducible evaluation, record several short test clips under different lighting conditions and include examples with a plain background, a background containing similarly coloured objects, and cloth colours close to the user's clothing. The present report does not claim a dataset size because these tests have not been supplied as a formal dataset.

5. Methodology / Proposed Approach

The application follows a frame-by-frame processing pipeline. The webcam frame is mirrored for a natural user view and resized to the configured working resolution. A background reference is captured while the scene is empty. During operation, the selected colour is segmented in HSV space. A foreground-change mask is computed by comparing the current frame with the reference background. Combining these masks reduces detections from static regions, although it cannot perfectly separate objects with similar colour or compensate for substantial scene changes.

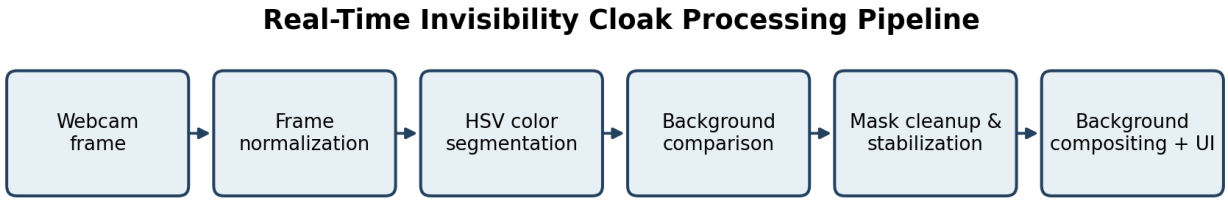


Figure 1. Processing pipeline for the real-time cloak prototype.

5.1 Algorithm / Pseudocode

11. Initialize webcam, MediaPipe Hands, selected colour, sensitivity, and interface state.
12. Capture a reference background while the scene is empty.
13. For each webcam frame: mirror and resize the frame.
14. Detect hand landmarks; map the index fingertip to the interface cursor.
15. Detect a pinch between the thumb tip and index fingertip; trigger a button only on a new pinch event.
16. Convert the frame to HSV and threshold the selected cloth colour.
17. If a background exists, calculate frame difference and combine it with the colour mask.
18. Apply morphological opening and closing, remove small regions, and smooth mask boundaries.
19. If the cloak is enabled, blend reference-background pixels into the masked region.
20. Render the interface and mask preview; read keyboard input and repeat until exit.

6. Techniques and Algorithms Used

Technique	Purpose in this project	Key consideration
BGR to HSV conversion	Separates hue and saturation from brightness for colour-based segmentation.	Thresholds are lighting- and camera-dependent.
Colour thresholding	Creates a binary mask for the selected cloth colour.	Other objects with similar colours may also be detected.

Frame differencing / background subtraction	Identifies pixels that differ from the captured reference scene.	Camera movement, shadows, and exposure changes create false foreground pixels.
Morphological opening	Removes small isolated mask noise.	Excessive filtering can remove thin cloth details.
Morphological closing	Fills small holes and joins nearby mask regions.	Nearby objects can merge into one region.
Gaussian smoothing	Softens mask boundaries for a less abrupt composite.	Too much blur can create colour leakage around edges.
Alpha compositing	Combines the current frame with the reference background using the mask.	Quality depends on mask accuracy and background alignment.
Hand landmark tracking	Tracks index fingertip and thumb tip for cursor and pinch interaction.	Hand occlusion and poor lighting can interrupt tracking.

Techniques such as grayscale-only processing, histogram equalization, edge detection, DFT/DCT, optical flow, and trained object detection are not central to the current implementation and are not claimed as implemented.

7. Implementation

Component	Implementation
Programming language	Python
Computer vision	OpenCV (cv2)
Numerical operations	NumPy
Hand tracking	MediaPipe Hands legacy Solutions API
Input	Webcam via OpenCV VideoCapture
Processing resolution	960 × 540 configured frame size
Interface	OpenCV-rendered buttons and cursor overlay
Interaction	Index-finger cursor; thumb–index pinch selects a button
Controls	Colour selection, background recapture, cloak toggle, mask preview, sensitivity keys, exit

7.1 Important Implementation Steps

21. Install Python and the required libraries in a virtual environment.
22. Open the webcam and confirm that frames can be read.
23. Capture the empty-scene background before bringing the cloth into view.
24. Use HSV thresholds for the selected cloth colour.
25. Apply frame differencing and morphological filtering to improve the mask.
26. Composite the captured background into the masked pixels.
27. Process hand landmarks and use pinch transitions to avoid repeated clicks while the fingers remain together.
28. Display the live output and provide keyboard fallbacks for key operations.

Compatibility note: the code uses mp.solutions.hands. A compatible MediaPipe release must be installed for this legacy API. If a different MediaPipe API is used, the hand-tracking integration must be updated accordingly.

8. Results and Discussion

8.1 Observed Functional Output

The prototype is designed to show the live camera view, a colour-segmentation mask, and a composite in which the detected cloth region is replaced by the corresponding pixels from the captured background. The hand interface supports cursor movement and pinch-based selection of on-screen options. The final visual quality depends on the selected colour range, the background reference, and the camera environment.

Test case	Expected / intended observation	Interpretation
Uniform coloured cloth against a contrasting background	Cloth region appears in the mask; background is composited into that region.	Best-case qualitative demonstration.
White cloth with a white wall	Wall may also be detected; background differencing may reduce static wall detections.	Known difficult case due to colour similarity.
Camera or subject-background movement during capture	Mask may show noise or misalignment.	Reference background should be captured with the camera fixed.
Low light or changing exposure	Colour thresholds and frame-difference mask may become unstable.	Lighting should be kept as consistent as possible.
Hand partly hidden from camera	Cursor tracking or pinch selection may become intermittent.	Keep the hand visible and avoid fast movement.

8.2 Intermediate Processing Outputs

- Original webcam frame: the mirrored live scene before compositing.
- Colour mask: a binary representation of pixels matching the selected colour and foreground-change conditions.
- Processed mask: a cleaned mask after morphological filtering and smoothing.
- Final composite: the live frame with captured-background pixels blended into the detected cloth region.
- Gesture interface: cursor location and on-screen button state driven by hand landmarks.

8.3 Quantitative Performance Measures

No measured accuracy, precision, recall, intersection-over-union (IoU), latency, or frames-per-second (FPS) values were provided for this submission. These values are therefore not invented. A quantitative evaluation can be completed by recording representative clips, manually annotating the cloth region, and comparing the generated mask with ground truth.

Metric	How to measure	Current status
FPS	Count processed frames over a timed interval.	Not measured
Latency	Measure elapsed time from frame capture to rendered output.	Not measured
IoU	Intersection(mask, ground truth) / Union(mask, ground truth).	Not measured
Precision / Recall	Compare mask pixels against manually annotated cloth pixels.	Not measured
Gesture selection success	Successful button activations / intended selections across repeated trials.	Not measured

For report finalization, insert genuine screenshots from the running application: (1) the original scene, (2) the cloth detection mask, (3) the final cloak composite, and (4) the hand-controlled interface with a selected button. Captions should identify the colour, lighting, and test condition. Avoid presenting a generated illustration as a measured experimental result.

9. Challenges and Limitations

- Colour ambiguity: white cloth, white walls, light clothing, and bright reflections can share similar HSV values.
- Background stability: the reference background becomes inaccurate if the camera moves or lighting changes significantly.
- Mask flicker: small changes in exposure, motion blur, and hand movement can change which pixels pass the thresholds.
- Edge quality: feathering reduces hard edges but can cause background leakage or halos if the mask boundary is inaccurate.

- Foreground differencing: moving shadows and camera noise can be interpreted as motion, while a cloth held still may be less distinguishable from the reference.
- Hand interaction: pinch detection depends on visible landmarks, hand orientation, and a suitable distance from the camera.
- Performance and compatibility: real-time speed depends on the device; MediaPipe API compatibility can vary by installed version.
- Evaluation: formal quantitative testing has not yet been performed, so claims are limited to the implemented functionality and qualitative observations.

10. Conclusion and Future Work

10.1 Conclusion

The project presents a real-time virtual invisibility-cloak prototype using Python, OpenCV, NumPy, and MediaPipe hand landmarks. It combines webcam acquisition, HSV colour segmentation, frame differencing, morphological mask cleanup, Gaussian smoothing, alpha compositing, and pinch-based interface interaction. The implementation demonstrates how image and video processing can create a recognisable visual effect and how hand landmarks can provide a hands-free control method. Its main limitation is that colour-based segmentation cannot reliably distinguish a cloth from other objects with similar colours under all lighting conditions.

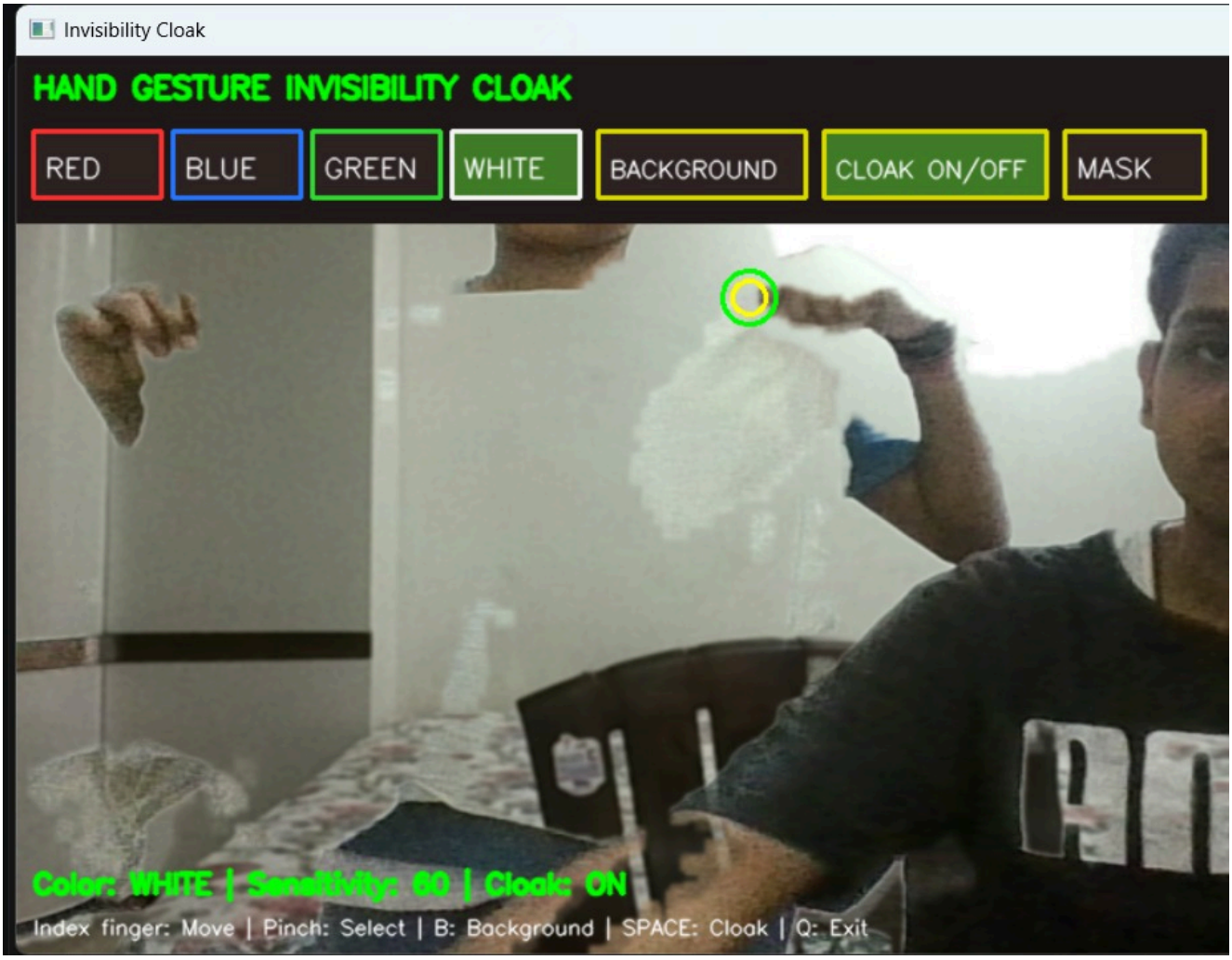
10.2 Learning Outcomes

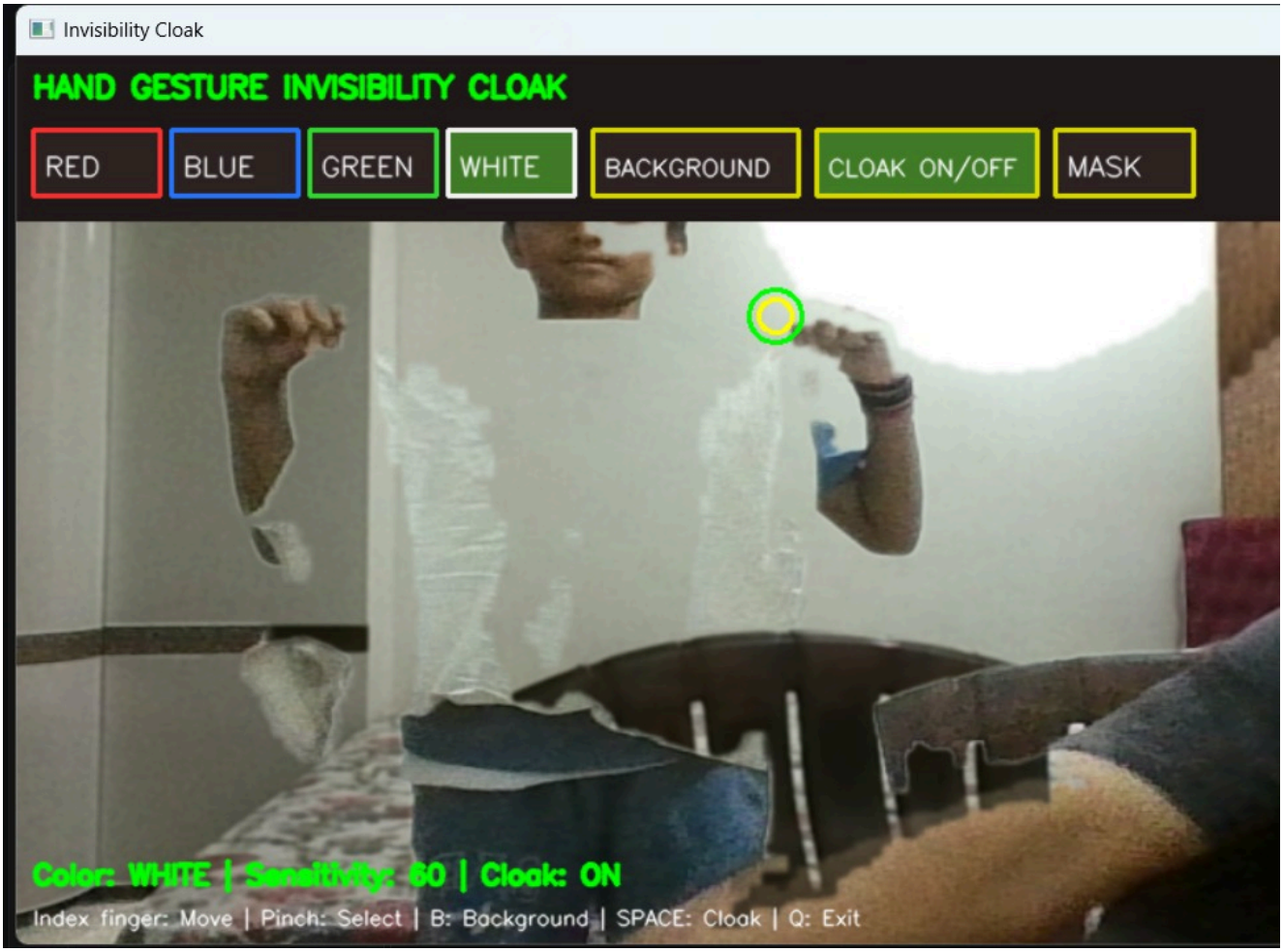
- Practical understanding of colour spaces and threshold-based segmentation.
- Experience with binary masks, morphology, frame differencing, and alpha compositing.
- Awareness of the effects of lighting, camera stability, and foreground-background similarity.
- Integration of real-time webcam processing with hand landmark tracking and GUI interaction.
- Importance of reproducible testing and honest reporting of measured versus qualitative results.

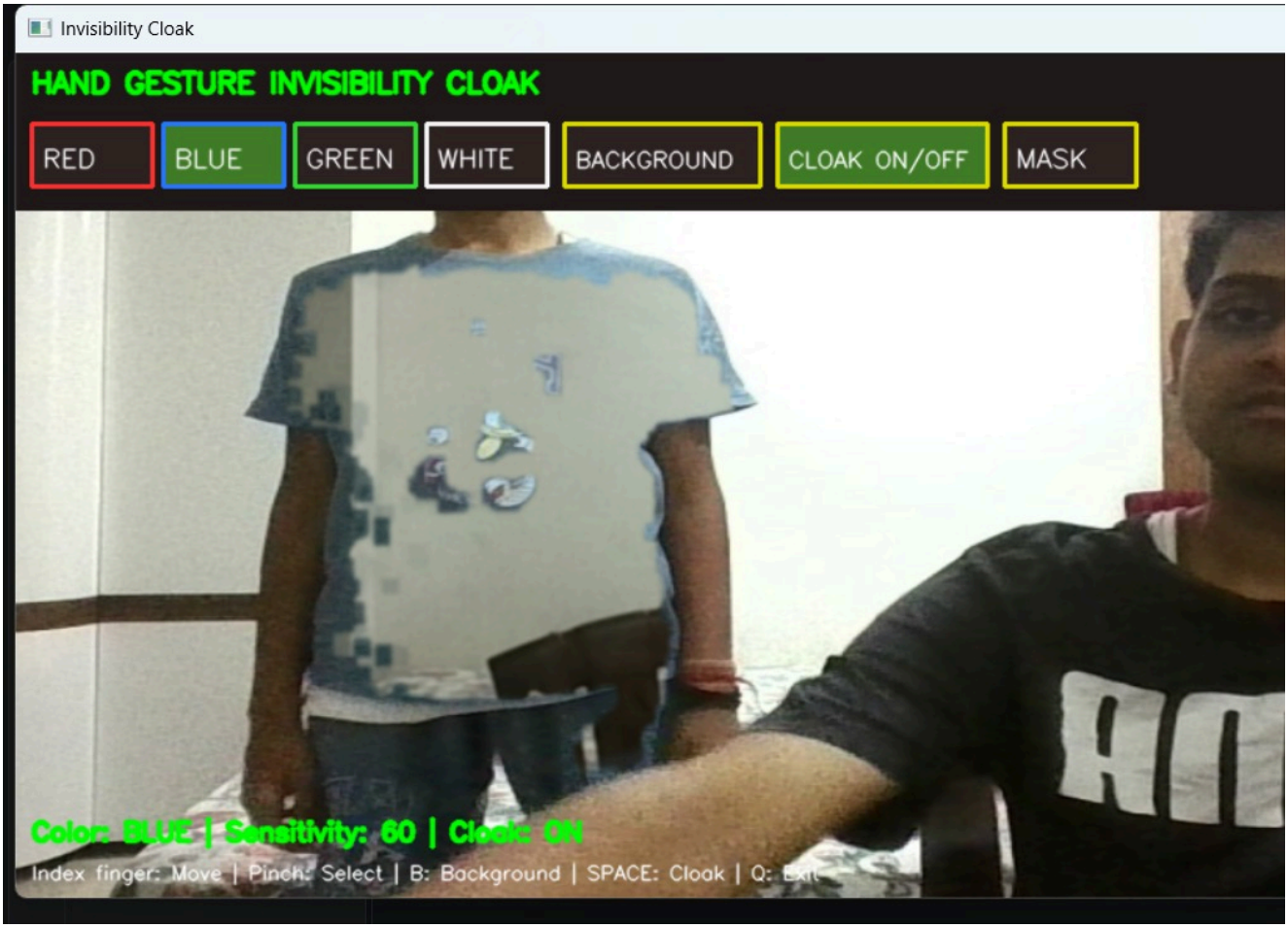
10.3 Future Work

- Replace simple colour segmentation with a cloth-region selection tool or a semantic segmentation model.
- Add a proper graphical sensitivity slider and hysteresis-based pinch detection.
- Improve mask stabilization and handle changing illumination more robustly.
- Record test videos and compute FPS, latency, IoU, precision, and recall.
- Add screenshot/video export and a more accessible interface.

Relevant Images -


Invisibility Cloak
HAND GESTURE INVISIBILITY CLOAK
RED
BLUE
GREEN
WHITE
BACKGROUND
CLOAK ON/OFF
MASK
Color: WHITE | Sensitivity: 60 | Cloak: ON
Index finger: Move | Pinch: Select | B: Background | SPACE: Cloak | Q: Exit


Invisibility Cloak
HAND GESTURE INVISIBILITY CLOAK
RED
BLUE
GREEN
WHITE
BACKGROUND
CLOAK ON/OFF
MASK
Color: WHITE | Sensitivity: 60 | Cloak: ON
Index finger: Move | Pinch: Select | B: Background | SPACE: Cloak | Q: Exit


Invisibility Cloak
HAND GESTURE INVISIBILITY CLOAK
RED
BLUE
GREEN
WHITE
BACKGROUND
CLOAK ON/OFF
MASK
Color: BLUE | Sensitivity: 60 | Cloak: ON
Index finger: Move | Pinch: Select | B: Background | SPACE: Cloak | Q:


Cloth Detection Mask

References

[1] G. Bradski, “The OpenCV Library,” Dr. Dobb’s Journal of Software Tools, 2000. OpenCV documentation: https://docs.opencv.org/

[2] OpenCV, “Changing Colorspaces — Object Tracking,” OpenCV-Python Tutorials. https://docs.opencv.org/4.x/df/d9d/tutorial_py_colorspaces.html

[3] OpenCV, “Morphological Transformations,” OpenCV-Python Tutorials. https://docs.opencv.org/4.x/d9/d61/tutorial_py_morphological_ops.html

[4] OpenCV, “Changing the contrast and brightness of an image,” OpenCV documentation and tutorials. https://docs.opencv.org/

[5] Google, “MediaPipe Hands,” MediaPipe documentation. https://developers.google.com/mediapipe/solutions/vision/hand_landmarker

[6] R. C. Gonzalez and R. E. Woods, Digital Image Processing, 4th ed., Pearson, 2018.

[7] OpenCV, “Background Subtraction,” OpenCV-Python Tutorials. https://docs.opencv.org/4.x/d1/dc5/tutorial_background_subtraction.html